Lab 2: The principles of Object Oriented Programming

Exercise 1: The Smart Thermostat System

Context: You are developing a prototype for a smart home automation company. You need to create a program that simulates a digital thermostat controlling the temperature of a room.

Your Task: Create a single Java file containing a class named `Thermostat`.

1. **Attributes:**
   - `roomName` (String)
   - `currentTemperature` (double)
   - `targetTemperature` (double)
2. **Methods:**
   - `initializeSettings(String name, double target)`: Assigns the room name and the target temperature. It should automatically set the `currentTemperature` to a default of `20.0` degrees.
   - `applyHeat(double amount)`: Increases the `currentTemperature` by the given amount.
   - `applyCooling(double amount)`: Decreases the `currentTemperature` by the given amount.
   - `displayStatus()`: Prints the current status of the thermostat. If `currentTemperature` is lower than `targetTemperature`, it should also print: `"[Status: HEATING ACTIVE]"`. Otherwise, it prints `"[Status: IDLE]"`.
3. **The Problem Setup (Main method):**
   - Inside the `main` method, create a `Thermostat` object.
   - Use the `initializeSettings` method to set up a `"Living Room"` with a target temperature of `24.0` degrees.
   - Display the initial status.
   - Apply heat by `2.5` degrees, then display the status again to check if the heating status updates correctly.

Exercise 2: E-Commerce Shipping Fees Calculator

Context: An online store needs an automated system to calculate the final shipping cost of a package based on its dimensions and the distance it travels.

Your Task: Create a Java class named `ShippingPackage`.

1. **Attributes:**
   - `packageName` (String)
   - `weight` (double) — in kilograms.
   - `distance` (int) — in kilometers.
2. **Methods:**

- `inputPackageData()`: This method must use the **`Scanner` class** to ask the user to type the package name, weight, and delivery distance from the keyboard.
- `calculateBaseCost()`: Returns a `double`. The base cost is calculated as: `weight * 1.5 + distance * 0.20`.
- `calculateExpressCost(boolean isExpress)`: An **overloaded** version or standard conditional method. If `isExpress` is `true`, add a fixed `$15.00` extra fee to the base cost. Otherwise, return just the base cost.
- `printInvoice()`: Displays the package details, the base shipping cost, and the final express shipping cost.

3. **The Problem Setup (Main method):**
   - Write a `main` method that instantiates one `ShippingPackage`.
   - Call `inputPackageData()` so the user can interact via the console.
   - Call `printInvoice()` to show the results based on the user's manual inputs.

Exercise 3: Fitness Tracker & Calorie Burner

Context: You are coding the logic for a wearable smart band. The band tracks a user's daily steps and calculates the total calories burned.

Your Task: Create a Java class named `FitnessTracker`.

1. **Attributes:**
   - `userName` (String)
   - `stepsCount` (int)
   - `caloriesBurnedPerStep` (double) — the amount of energy spent per step (e.g., `0.04` calories).
2. **Methods:**
   - `registerUser(String name)`: Sets the user's name, sets `stepsCount` to `0`, and sets `caloriesBurnedPerStep` to `0.04`.
   - `walk(int steps)`: Adds the number of steps taken to the `stepsCount` attribute. **Validation:** If the input steps are negative, display an error message: `"Invalid movement!"` and do not change the data.
   - `getTotalCalories()`: Computes and returns the total calories burned (`stepsCount * caloriesBurnedPerStep`).
   - `resetDailyData()`: Resets the `stepsCount` back to `0` (simulating a new day).
   - `showReport()`: Displays the user's name, total steps, and total calories burned.
3. **The Problem Setup (Main method):**
   - In the `main` method, create a tracker for `"John"`.
   - Simulate a morning walk of `5000` steps.
   - Simulate an afternoon run of `3500` steps.
   - Print the workout report.
   - Reset the daily data, and print the report one last time to verify it returns to `0`.

Exercise :

We propose to create the slot machine game. It's a game of chance played on a machine with three spinning wheels, each displaying a symbol (for example, apricot, banana, star).
When the wheels stop, if all three wheels have the same symbol, you win.

We propose:
To create this game of chance.

We will represent the symbols with numbers (e.g., 1, 2, and 3).

The possible winnings will be as follows: 10 points for two identical symbols, 100 points for three identical symbols, and 0 points otherwise.

Tasks to complete:

- a Wheel class characterized by a Symbol attribute and the following methods:

  - spin(): Generates a random number and stores it in the symbol attribute displayed on the wheel.

Knowing that: Math.Random() is a method to generate a random number between 0 and 1.

  - getSymbol(): returns the value of the symbol stored on the current wheel.

- A BanditManchot class which includes:
  - an array of 3 Wheels,
  - a constructor to create this array, and
  - a play() method: which will display the 3 wheels and return the game's payout: 10 points for two identical symbols, 100 points for 3 identical symbols, and 0 points otherwise.

- A TestJeu class to test this game once and displays one of the following messages depending on the result:

"You've won! Your score is 100 points" or
"You're about to succeed! Your score is 10 points" or
"Sorry! See you next time."